



Cairo University
Faculty of Computers and Artificial Intelligence
Information Systems Department

Introduction to Database
Lab 3
DDL + DML (Foreign Keys +
Select, Joins)

Part (1) DDL (Foreign Keys)

Using the schema in the 1st Lab creating the foreign keys and inserting data in all tables:

- Delete the data from table **Course**.
- Deleting a parent row (Course) could leave rows on a child table (Register) pointing to a non-existent ID. This leads to broken reports and data corruption.

```
Messages
Msg 547, Level 16, State 0, Line 1
The DELETE statement conflicted with the REFERENCE constraint "FK_register_CrsCod_4222D4EF". The conflict occurred in database "UNIVERSITY", table "dbo.register", column 'CrsCode'.
The statement has been terminated.

Completion time: 2026-03-06T03:57:51.2020017+02:00
```

- Depending on the case, the constraint could be redefined to the following scenarios:

Action	ON DELETE Behavior	ON UPDATE Behavior
CASCADE	Deletes child rows	Updates child keys
SET NULL	Child foreign key becomes NULL	Child foreign key becomes NULL
NO ACTION	Default; blocks delete	Default; blocks update
SET DEFAULT	Sets child to default value	Sets child to default value

Syntax

```
ALTER TABLE <Table_Name>
ALTER CONSTRAINT <Constraint_Name>
FOREIGN KEY <Col> REFERENCES <Table>(<Col>)
ON DELETE <Action>
```

Action	Result in Register Table	Use Case
CASCADE	Row is deleted	Registrations are useless without the course.
SET NULL	CourseID becomes NULL	Need to know a student registered, even if the course is gone.
SET DEFAULT	CourseID becomes Default	Move students to a "General" or "TBD" category.
NO ACTION	Deletion is blocked (Error)	Prevent course deletion if students are still enrolled.

Part (2) DML1 (Select Statement)

Simple Select Statements SELECT <Column list> FROM <table names> [WHERE <Condition>]	Example 1: (using northwind database): ❖ Retrieve all information of all products SELECT * FROM [dbo].[products]
Example 2: ❖ Retrieve Product id and Product name of all products SELECT ProductID, ProductName FROM products	Example 3: ❖ Get all PRODUCTS where their names contain the letter H SELECT * FROM products where ProductName like '%h%'
Example 4: (Using DISTINCT Keyword) ❖ Get all unique CATOGRIES of the products SELECT DISTINCT CategoryID FROM [dbo].[products] ❖ Get all PRODCUTS where their Category is either 1 or 2 SELECT * FROM [dbo].[products] WHERE categoryid=1 or categoryid= 2	

Another way:

```
SELECT *  
FROM [dbo].[products]  
WHERE categoryid IN (1,2)
```

Part (3) DML 2 (Joins)

SQL Joins:

SQL Joins are used to fetch/retrieve data from two or more data tables, based on a join condition. A join condition is a relationship among some columns in the data tables that take part in SQL join. Basically, database tables are related to each other with keys. We use this keys relationship in SQL Joins.

Types of Joins:

- **Inner Join:** returns only those tuples that match the join condition.
- **Outer Join:**
 - **Left outer Join:** every tuple in the left table must appear in the result; if it does not have a matching tuple; it is padded with NULL values for the attributes of the right table.
 - **Right Outer Join:** every tuple in the right table must appear in the result; if it does not have a matching tuple, it is padded with NULL values for the attributes of the left table.
 - **FULL Outer Join:** Keeps all tuples in both the left and the right relations when no matching tuples are found, padding them with NULL values as needed.
- **Cross Join:** Specifies the CARTESIAN PRODUCT operation, it generates all possible tuple combinations.

Join Examples(Using Northwind DB):

Example 1:

Get the names, prices and names of companies that supply these products.

```
SELECT Products.ProductName,Products.UnitPrice,Suppliers.CompanyName  
FROM Products inner join Suppliers  
On Products.SupplierID = Suppliers.SupplierID  
--OR  
SELECT Products.ProductName,Products.UnitPrice,Suppliers.CompanyName  
FROM Products, Suppliers WHERE Products.SupplierID =  
Suppliers.SupplierID
```

Example 2:

Get the names of products categories and the name of the company that supply these categories.

```
Select Categories.CategoryName,Suppliers.CompanyName
from Categories inner join Products
on Categories.CategoryID =Products.CategoryID
inner join Suppliers
on Products.SupplierID=Suppliers.SupplierID;
```

--OR

```
SELECT Categories.CategoryName, Suppliers.CompanyName
FROM Categories, Products, Suppliers
WHERE Suppliers.SupplierID = Products.SupplierID
      AND Products.CategoryID = Categories.CategoryID
```

Example 3:

Get names of all the customers and if they place any orders show their order date.

```
SELECT Customers.CompanyName, Orders.OrderDate
FROM Orders RIGHT OUTER JOIN Customers
ON Orders.CustomerID = Customers.CustomerID
```

OR

```
SELECT Customers.CompanyName, Orders.OrderDate
FROM Customers LEFT OUTER JOIN Orders
ON Customers.CustomerID=Orders.CustomerID
```

Example 4:

Get names of all products and all categories ,showing which products belong to which categories if any:

```
SELECT Products.ProductName, Categories.CategoryName
FROM Products FULL OUTER JOIN Categories
ON Products.CategoryID =Categories.CategoryID
```

Note: you need to insert a new Category and a new product to be able to know the changes at the results

- `INSERT INTO Categories (CategoryName) Values('New Category')`

- `INSERT INTO Products (ProductName,Discontinued) Values ('New Product',0)`

Example 5:

Get all combinations of Products and Suppliers ,Mentioning the names of products and the suppliers.

```
Select Products.ProductName,Suppliers.CompanyName  
from Products CROSS JOIN Suppliers
```